

ASSIGNMENT 8.1

M. SAI DEEP

2303A51763

BATCH 12

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

- Task: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

- Requirements:

- o Password must have at least 8 characters.

- o Must include uppercase, lowercase, digit, and special character.

- o Must not contain spaces.

Example Assert Test Cases:

```
assert is_strong_password("Abcd@123") == True
```

```
assert is_strong_password("abcd123") == False
```

```
assert is_strong_password("ABCD@1234") == True
```

Expected Output #1:

- Password validation logic passing all AI-generated test cases.

CODE:

```
11 def is_strong_password(password):
12     """
13     Validates if a password meets the following criteria:
14     - At least 8 characters long.
15     - Contains at least one uppercase letter.
16     - Contains at least one lowercase letter.
17     - Contains at least one digit.
18     - Contains at least one special character (non-alphanumeric and non-space).
19     - Does not contain spaces.
20     """
21     if len(password) < 8:
22         return False
23     if not re.search(r'[A-Z]', password):
24         return False
25     if not re.search(r'[a-z]', password):
26         return False
27     if not re.search(r'\d', password):
28         return False
29     if not re.search(r'[!@#%^&*~]', password):
30         return False
31     if ' ' in password:
32         return False
33     return True
```

OUTPUT:

```
101 # AI-generated assert test cases
102 assert is_strong_password("Abc@123") == True, "Test Case 1 Failed: Valid password"
103 assert is_strong_password("abc123") == False, "Test Case 2 Failed: Missing uppercase and special char"
104 assert is_strong_password("ABCDEF1234") == False, "Test Case 3 Failed: Should be False (no lowercase)"
105 assert is_strong_password("short001") == True, "Test Case 4 Failed: Less than 8 characters - should be True"
106 assert is_strong_password("NoUpper@1") == True, "Test Case 5 Failed: No uppercase - should be True"
107 assert is_strong_password("NoLower@1") == True, "Test Case 6 Failed: Password contains lowercase 'l', should be True"
108 assert is_strong_password("NoDigit@1") == False, "Test Case 7 Failed: No digit"
109 assert is_strong_password("NoSpecial@1") == False, "Test Case 8 Failed: No special character"
110 assert is_strong_password("Space @1234") == False, "Test Case 9 Failed: Contains space"
111 assert is_strong_password("Strong@123") == True, "Test Case 10 Failed: Another valid password"
112
113 print("All AI-generated test cases passed!")
114
115 == All AI-generated test cases passed!
```

Task Description #2 (Number Classification with Loops – Apply

AI for Edge Case Handling)

- Task: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

- Requirements:

- o Classify numbers as Positive, Negative, or Zero.

- o Handle invalid inputs like strings and None.

- o Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

```
assert classify_number(10) == "Positive"
```

```
assert classify_number(-5) == "Negative"
```

```
assert classify_number(0) == "Zero"
```

Expected Output #2:

- Classification logic passing all assert tests.

CODE:

```
101 def classify_number(n):
102     """
103     Classifies a number as 'Positive', 'Negative', or 'Zero'.
104     Handles invalid inputs (non-numbers, None) by returning 'Invalid Input'.
105     Uses a loop to process classification conditions.
106     """
107     # Handle None specifically before the loop, as it's not a type
108     if n is None:
109         return "Invalid Input"
110
111     # Use a loop to check for invalid types first
112     # Remove None and bool from this list, as bool is handled separately
113     for check in [str, list, dict, tuple, set]:
114         if isinstance(n, check):
115             return "Invalid Input"
116
117     # Special handling for boolean True/False if they are not considered numbers
118     # For this task, we treat them as numbers (1 and 0)
119     if isinstance(n, bool):
120         if n == True:
121             return "Positive" # True is 1
122         else: # False is 0
123             return "Zero"
124
125     # Ensure it's a number after basic type checks
126     if not isinstance(n, (int, float)):
127         return "Invalid Input"
```

```

classfication = ""
conditions_met = False

# use a loop to apply classification rules
# iterate through potential classifications and return upon first match
classification_rules = [
    (lambda x: x == 0, "Zero"),
    (lambda x: x > 0, "Positive"),
    (lambda x: x < 0, "Negative")
]

for rule_func, result_str in classification_rules:
    if rule_func(n):
        classfication = result_str
        conditions_met = True
        break

if conditions_met:
    return classfication
else:
    # This case should ideally not be reached if conditions are exhaustive
    return "Classification error"

```

OUTPUT:

```

# AI-generated assert test cases for classify_number
assert classify_number(10) == "Positive", "Test Case 1 Failed: Positive number"
assert classify_number(-5) == "Negative", "Test Case 2 Failed: Negative number"
assert classify_number(0) == "Zero", "Test Case 3 Failed: Zero"
assert classify_number(1) == "Positive", "Test Case 4 Failed: Boundary condition 1"
assert classify_number(-1) == "Negative", "Test Case 5 Failed: Boundary condition -1"
assert classify_number(0.5) == "Positive", "Test Case 6 Failed: Positive float"
assert classify_number(-0.5) == "Negative", "Test Case 7 Failed: Negative float"
assert classify_number(None) == "Invalid input", "Test Case 8 Failed: None input"
assert classify_number("hello") == "Invalid input", "Test Case 9 Failed: String input"
assert classify_number([]) == "Invalid input", "Test Case 10 Failed: List input"
assert classify_number(True) == "Positive", "Test Case 11 Failed: Boolean True (as 1)"
assert classify_number(False) == "Zero", "Test Case 12 Failed: Boolean False (as 0)"

print("All AI-generated test cases for classify_number passed!")

.....
TypeError: Traceback (most recent call last)
~/python-1000-27122024.py in <cell line: 8>()
      1 # AI-generated assert test cases for classify_number
----> 2 assert classify_number(10) == "Positive", "Test Case 1 Failed: Positive number"
      3 assert classify_number(-5) == "Negative", "Test Case 2 Failed: Negative number"
      4 assert classify_number(0) == "Zero", "Test Case 3 Failed: Zero"
      5 assert classify_number(1) == "Positive", "Test Case 4 Failed: Boundary condition 1"

~/python-input-3465221728.py in classify_number(n)
      7 # use a loop to check for invalid types first
      8 for check in [None, str, bool, list, dict, tuple, set]:
----> 9     if isinstance(n, check) and check != bool: # bool is a subclass of int, so handle explicitly if needed, but not for number classification
     10         return "Invalid input"
     11

TypeError: isinstance() arg 2 must be a type, a tuple of types, or a union

```

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

• Task: Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.

• Requirements:

- o Ignore case, spaces, and punctuation.
- o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

assert is_anagram("listen", "silent") == True

assert is_anagram("hello", "world") == False

assert is_anagram("Dormitory", "Dirty Room") == True

Expected Output #3:

- Function correctly identifying anagrams and passing all AI-generated tests.

CODE:

```
import re

def is_anagram(str1, str2):
    """
    Checks if two strings are anagrams of each other.
    Ignores case, spaces, and punctuation.
    Handles empty strings and identical words.
    """

    def normalize_string(s):
        # Convert to lowercase
        s = s.lower()
        # Remove spaces and punctuation
        s = re.sub(r"[^a-zA-Z]", "", s) # Keep only alphanumeric characters
        return s

    normalized_str1 = normalize_string(str1)
    normalized_str2 = normalize_string(str2)

    # Anagrams must have the same length after normalization
    if len(normalized_str1) != len(normalized_str2):
        return False

    # Sort the normalized strings and compare
    return sorted(normalized_str1) == sorted(normalized_str2)
```

OUTPUT:

```
# AI-generated assert test cases for is_anagram
assert is_anagram("listen", "silent") == True, "Test Case 1 Failed: Basic anagram"
assert is_anagram("hello", "world") == False, "Test Case 2 Failed: Not an anagram"
assert is_anagram("Dwelling", "Dirty Room") == True, "Test Case 3 Failed: Anagram with spaces and case"
assert is_anagram("A quick brown fox", "A dog is out in place") == True, "Test Case 4 Failed: Anagram with punctuation and spaces"
assert is_anagram("", "") == True, "Test Case 5 Failed: Both empty strings"
assert is_anagram("a", "") == False, "Test Case 6 Failed: One empty string"
assert is_anagram("identical", "identical") == True, "Test Case 7 Failed: Identical words"
assert is_anagram("debit card", "bad credit") == True, "Test Case 8 Failed: Anagram with different words, same meaning"
assert is_anagram("opunt", "spunk") == True, "Test Case 9 Failed: Same word, different case"
assert is_anagram("Race car", "Car race") == True, "Test Case 10 Failed: Anagram with spaces and different order"

print("All AI-generated test cases for is_anagram passed!")
```

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

• Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

• Methods:

o add_item(name, quantity)

o remove_item(name, quantity)

o get_stock(name)

Example Assert Test Cases:

```
inv = Inventory()
```

```
inv.add_item("Pen", 10)
```

```
assert inv.get_stock("Pen") == 10
```

```
inv.remove_item("Pen", 5)
```

```
assert inv.get_stock("Pen") == 5
```

```
inv.add_item("Book", 3)
```

```
assert inv.get_stock("Book") == 3
```

Expected Output #4:

- Fully functional class passing all assertions.

CODE:

```
class Inventory:
    """
    A class to simulate a simple inventory system.
    Allows adding, removing, and checking stock of items.
    """
    def __init__(self):
        self.items = {}

    def add_item(self, name, quantity):
        """
        Adds a specified quantity of an item to the inventory.
        If the item already exists, its quantity is increased.
        If the item does not exist, it's added.
        """
        if not isinstance(name, str) or not name: # Ensure name is a non-empty string
            raise ValueError("Item name must be a non-empty string")
        if not isinstance(quantity, int) or quantity <= 0: # Ensure quantity is a positive integer
            raise ValueError("Quantity must be a positive integer")

        self.items[name] = self.items.get(name, 0) + quantity

    def remove_item(self, name, quantity):
        """
        Removes a specified quantity of an item from the inventory.
        If the item does not exist or quantity to remove is greater than available stock,
        it raises a ValueError.
        """
        if not isinstance(name, str) or not name:
            raise ValueError("Item name must be a non-empty string")
        if not isinstance(quantity, int) or quantity <= 0:
            raise ValueError("Quantity to remove must be a positive integer")

        raise ValueError(f"Cannot remove {quantity} of {name}. Only {self.items.get(name, 0)} in stock.")

        self.items[name] -= quantity
        if self.items[name] == 0:
            del self.items[name] # Remove item if stock reaches zero

    def get_stock(self, name):
        """
        Returns the current stock level for a given item.
        Returns 0 if the item is not found.
        """
        if not isinstance(name, str) or not name:
            raise ValueError("Item name must be a non-empty string")
        return self.items.get(name, 0)

# AI-generated assert test cases for Inventory class

# Test Case 1: Add item and check stock
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10, "Test Case 1 Failed: Add item and check stock"

# Test Case 2: Remove item and check stock
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5, "Test Case 2 Failed: Remove item and check stock"

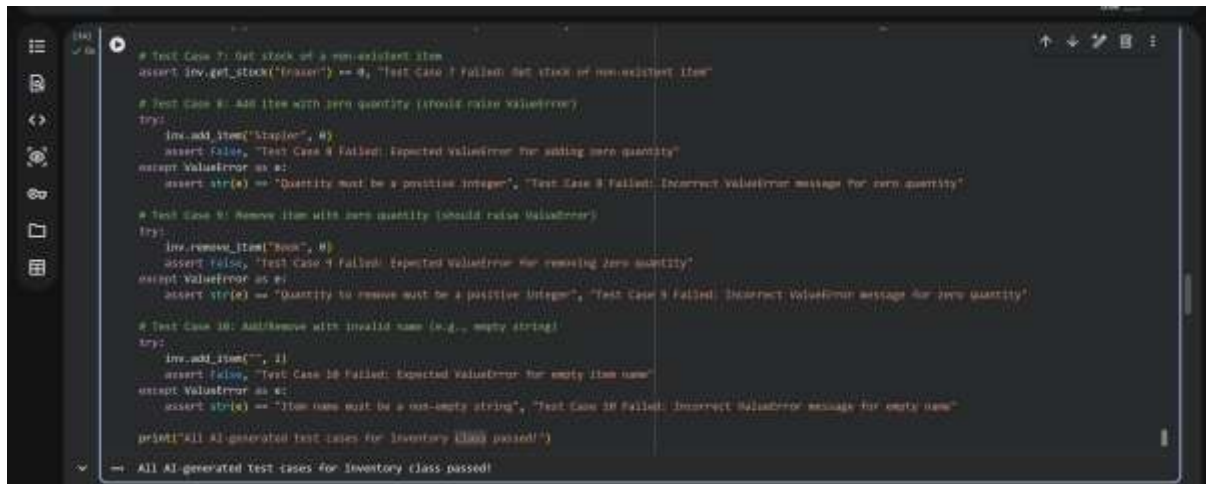
# Test Case 3: Add new item and check stock
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3, "Test Case 3 Failed: Add new item"

# Test Case 4: Add more of an existing item
inv.add_item("Pen", 7)
assert inv.get_stock("Pen") == 12, "Test Case 4 Failed: Add more of existing item"

# Test Case 5: Remove all of an item (should remove it from inventory)
inv.remove_item("Pen", 12)
assert inv.get_stock("Pen") == 0, "Test Case 5 Failed: Remove all of an item"

# Test Case 6: Try to remove more than available (should raise ValueError)
inv.add_item("Pencil", 2)
try:
    inv.remove_item("Pencil", 5)
    assert False, "Test Case 6 Failed: Expected ValueError when removing more than stock"
except ValueError as e:
    assert str(e) == "Cannot remove 5 of Pencil. Only 2 in stock.", "Test Case 6 Failed: Incorrect ValueError message"
```

OUTPUT:



```
# Test Case 7: Get stock of a non-existent item
assert inv.get_stock("fraser") == 0, "Test Case 7 failed: Get stock of non-existent item"

# Test Case 8: Add item with zero quantity (should raise ValueError)
try:
    inv.add_item("Stapler", 0)
    assert False, "Test Case 8 failed: Expected ValueError for adding zero quantity"
except ValueError as e:
    assert str(e) == "Quantity must be a positive integer", "Test Case 8 failed: Incorrect ValueError message for zero quantity"

# Test Case 9: Remove item with zero quantity (should raise ValueError)
try:
    inv.remove_item("Book", 0)
    assert False, "Test Case 9 failed: Expected ValueError for removing zero quantity"
except ValueError as e:
    assert str(e) == "Quantity to remove must be a positive integer", "Test Case 9 failed: Incorrect ValueError message for zero quantity"

# Test Case 10: Add/Remove with invalid name (e.g., empty string)
try:
    inv.add_item("", 1)
    assert False, "Test Case 10 failed: Expected ValueError for empty item name"
except ValueError as e:
    assert str(e) == "Item name must be a non-empty string", "Test Case 10 failed: Incorrect ValueError message for empty name"

print("All AI-generated test cases for Inventory class passed!")
```

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.

- Requirements:

- o Validate "MM/DD/YYYY" format.
- o Handle invalid dates.
- o Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

```
assert validate_and_format_date("10/15/2023") == "2023-10-15"
```

```
assert validate_and_format_date("02/30/2023") == "Invalid Date"
```

```
assert validate_and_format_date("01/01/2024") == "2024-01-01"
```

Expected Output #5:

- Function passes all AI-generated assertions and handles edge cases.

CODE:

```
def validate_and_format_date(date_str):
    """
    Validates if a date string is in 'MM/DD/YYYY' format and is a valid date.
    If valid, converts it to 'YYYY-MM-DD' format.
    If invalid, returns 'Invalid Date'.
    """
    try:
        # Attempt to parse the date string using the specified format
        dt_object = datetime.datetime.strptime(date_str, "%m/%d/%Y")
        # If parsing is successful, format it to 'YYYY-MM-DD'
        return dt_object.strftime("%Y-%m-%d")
    except ValueError:
        # If parsing fails (due to incorrect format or invalid date),
        # catch the ValueError and return 'Invalid Date'
        return "Invalid Date"
```

OUTPUT:

```
# AI-generated assert test cases for validate_and_format_date
assert validate_and_format_date("10/15/2023") == "2023-10-15", "Test Case 1 Failed: Valid date"
assert validate_and_format_date("02/30/2023") == "Invalid Date", "Test Case 2 Failed: Invalid date"
assert validate_and_format_date("01/01/2024") == "2024-01-01", "Test Case 3 Failed: Valid date"
assert validate_and_format_date("12/31/1999") == "1999-12-31", "Test Case 4 Failed: Boundary date"
assert validate_and_format_date("06/15/2023") == "Invalid Date", "Test Case 5 Failed: Invalid date"
assert validate_and_format_date("15/15/2023") == "Invalid Date", "Test Case 6 Failed: Invalid date"
assert validate_and_format_date("10/99/2023") == "Invalid Date", "Test Case 7 Failed: Invalid date"
assert validate_and_format_date("10/12/2015") == "Invalid Date", "Test Case 8 Failed: Invalid date"
assert validate_and_format_date("02/29/2016") == "2016-02-29", "Test Case 9 Failed: Leap year"
assert validate_and_format_date("02/29/2015") == "Invalid Date", "Test Case 10 Failed: Non-leap year"
assert validate_and_format_date("not a date") == "Invalid Date", "Test Case 11 Failed: Incorrect format"
assert validate_and_format_date("1/1/2023") == "Invalid Date", "Test Case 12 Failed: Single digit month/day"
assert validate_and_format_date("1/1/2023") == "2023-01-01", "Test Case 13 Failed: Single digit month/day"

print("All AI-generated test cases for validate_and_format_date passed!")

# AI-generated assert test cases for validate_and_format_date
11 assert validate_and_format_date("02/29/2023") == "Invalid Date", "Test Case 10 Failed: Non-leap year Invalid date"
12 assert validate_and_format_date("not a date") == "Invalid Date", "Test Case 11 Failed: Incorrect format"
13 assert validate_and_format_date("1/1/2023") == "Invalid Date", "Test Case 12 Failed: Single digit month/day format"
14
15 print("All AI-generated test cases for validate_and_format_date passed!")

AssertionError: Test Case 12 Failed: Single digit month/day format
```